Basic Text Processing

Regular Expressions

Regular expressions are used everywhere

- Part of every text processing task
  - Not a general NLP solution (for that we use large NLP systems we will see in later lectures)
  - But very useful as part of those systems (e.g., for pre-processing or text formatting)
- Necessary for data analysis of text data
- A widely used tool in industry and academics

Regular expressions

A formal language for specifying text strings

How can we search for mentions of these cute animals in text?

- woodchuck
- woodchucks
- Woodchuck
- Woodchucks
- Groundhog
- groundhogs

Regular Expressions: Disjunctions

Letters inside square brackets []

Pattern	Matches
`[wW]oodchuck`	Woodchuck, woodchuck
`[1234567890]`	Any one digit

Ranges using the dash `[A-Z]`

Pattern	Matches	
`[A-Z]`	An upper case letter	`Drenched Blossoms`
`[a-z]`	A lower case letter	`my beans were impatient`
`[0-9]`	A single digit	`Chapter 1: Down the Rabbit Hole`

Regular Expressions: Negation in Disjunction

Carat as first character in [] negates the list

- Note: Carat means negation only when it's first in []
- Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	Oyfn pripetchik
[^Ss]	Neither ‘S’ nor ‘s’	I have no exquisite reason”
[^.]	Not a period	Our resident Djinn
[e^]	Either e or ^	Look up ^ now

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
`\d`	`[0-9]`	Any digit	`Fahreneit 451`
`\D`	`[^0-9]`	Any non-digit	`Blue Moon`
`\w`	`[a-ZA-Z0-9_]`	Any alphanumeric or _	`Daiyu`
`\W`	`[^\w]`	Not alphanumeric or _	`Look!`
`\s`	`[ \r\t\n\f]`	Whitespace (space, tab)	`Look␣up`
`\S`	`[^\s]`	Not whitespace	`Look up`

Regular Expressions: More Disjunction

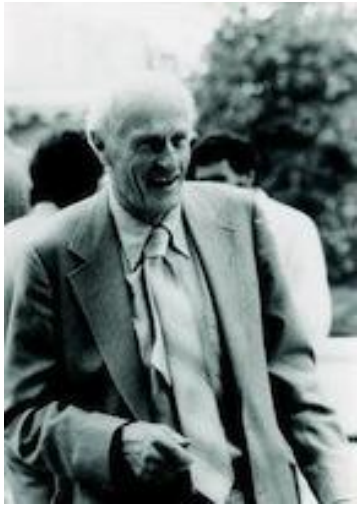
Groundhog is another name for woodchuck!

The pipe symbol | for disjunction

Pattern	Matches
`groundhog|woodchuck`	woodchuck
`yours|mine`	yours
`a|b|c`	= [abc]
`[gG]roundhog|[Ww]oodchuck`	Woodchuck

Wildcards, optionality, repetition: . ? * +

Pattern	Matches	Examples
beg.n	Any char	begin begun beg3n beg n
woodchucks?	Optional s	woodchuck woodchucks
to*	0 or more of previous char	t to too tooo
to+	1 or more of previous char	to too tooo toooo



Stephen C Kleene

Kleene *, Kleene +

Regular Expressions: Anchors ^ $

Pattern	Matches
^[A-Z]	Palo Alto
^[^A-Za-z]	1 "Hello"
\.$	The end.
.$	The end? The end!

A note about Python regular expressions

- Regex and Python both use backslash "\" for special characters. You must type extra backslashes!
  - `"\\d+"` to search for 1 or more digits
  - `"\n"` in Python means the "newline" character, not a "slash" followed by an "n". Need `"\\n"` for two characters.
- Instead: use Python's **raw string notation** for regex:
  - `r"[tT]he"`
  - `r"\d+"` matches one or more digits
    - instead of `"\\d+"`

The iterative process of writing regex's

Find me all instances of the word “the” in a text.

`the`

Misses capitalized examples

`[tT]he`

Incorrectly returns `other` or `Theology`

`\W[tT]he\W`

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched (The)

   False negatives

2. Matching strings that we should not have matched (there, then, other)

   False positives

Characterizing work on NLP

In NLP we are always dealing with these kinds of errors.

Reducing the error rate for an application often involves two antagonistic efforts:

- Increasing coverage (or *recall*) (minimizing false negatives).
- Increasing accuracy (or *precision*) (minimizing false positives)

Regular expressions play a surprisingly large role

Widely used in both academics and industry

1. Part of most text processing tasks, even for big neural language model pipelines
   - including text formatting and pre-processing
2. Very useful for data analysis of any text data

```
import re

# text from wikipedia
text = """American International University-Bangladesh commonly known
by its acronym AIUB, is a private research university in Dhaka,
Bangladesh. This university was established in 1994. It offers several
degree programs at the undergraduate and graduate levels from its five
faculties."""

# .span() returns a tuple containing the start-, and end positions of
the match.
# .string returns the string passed into the function
# .group() returns the part of the string where there was a match

capitalized_word_matches = re.search(r"[A-Z]\w+", text)
print(capitalized_word_matches)
print(capitalized_word_matches.span())
print(capitalized_word_matches.string)
print(capitalized_word_matches.group())
```

```
import re

# text from wikipedia
text = """American International University-Bangladesh commonly known
by its acronym AIUB, is a private research university in Dhaka,
Bangladesh. This university was established in 1994. It offers several
degree programs at the undergraduate and graduate levels from its five
faculties."""

capitalized_word_matches = re.search(r"[A-Z]\w+", text)
print(capitalized_word_matches)
print(capitalized_word_matches.span())
print(capitalized_word_matches.string)
print(capitalized_word_matches.group())
```

Output

```
<re.Match object; span=(0, 8), match='American'>
(0, 8)
American International University-Bangladesh commonly known by its acronym AIUB,
is a private research university in Dhaka, Bangladesh. This university was
established in 1994.It offers several degree programs at the undergraduate and
graduate levels from its five faculties.
American
```

```
import re

text = """American International University-Bangladesh
commonly known by its acronym AIUB, is a private research
university in Dhaka, Bangladesh. This university was
established in 1994. It offers several degree programs at the
undergraduate and graduate levels from its five faculties."""

capitalized_word_list = re.findall(r"[A-Z]\w+", text)
print(capitalized_word_list)
```

```
import re

text = """American International University-Bangladesh
commonly known by its acronym AIUB, is a private research
university in Dhaka, Bangladesh. This university was
established in 1994. It offers several degree programs at the
undergraduate and graduate levels from its five faculties."""

capitalized_word_list = re.findall(r"[A-Z]\w+", text)
print(capitalized_word_list)
```

Output

```
['American', 'International', 'University', 'Bangladesh',
'AIUB', 'Dhaka', 'Bangladesh', 'This', 'It']
```

```
import re

text = 'Today Sehri time in Dhaka
is 05:03 am. Today Iftar time in
Dhaka is 6:03 PM.’

text = text.lower()
sentence_pattern = r"[.]"
sentence_list =
re.split(sentence_pattern, text)
sentence_list.remove('')
print(sentence_list)
```

```
for sentence in sentence_list:
  sentence = sentence.strip()
  time_pattern = r"\d+:\d+ [ap]m"
  time = re.findall(time_pattern,
sentence, re.IGNORECASE)
  if time is not None:
    print(sentence)
    if "sehri" in sentence:
      print("Sehri: ", time[0])
    elif "iftar" in sentence:
      print("Ifter: ", time[0])
    else:
      print("Unknown: ", time[0])
```

```
import re

text = 'Today Sehri time in Dhaka is 05:03 am. Today Iftar time in Dhaka is 6:03 PM.'

text = text.lower()
sentence_pattern = r"[.]"
sentence_list = re.split(sentence_pattern, text)
sentence_list.remove('')
print(sentence_list)
```

```
for sentence in sentence_list:
  sentence = sentence.strip()
  time_pattern = r"\d+:\d+ [ap]m"
  time = re.findall(time_pattern, sentence, re.IGNORECASE)
  if time is not None:
    print(sentence)
    if "sehri" in sentence:
      print("Sehri: ", time[0])
    elif "iftar" in sentence:
      print("Ifter: ", time[0])
    else:
      print("Unknown: ", time[0])
```

Output

```
['today sehri time in dhaka is 05:03 am', ' today iftar time in dhaka is 6:03 pm']
today sehri time in dhaka is 05:03 am
Sehri:  05:03 am
today iftar time in dhaka is 6:03 pm
Ifter:  6:03 pm
```

References

- Daniel Jurafsky and James H. Martin. 2024. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition.